



Assignment 2

Course: COL334

Submitted By

Preksha Rathal : 2024CS50987
Kesar Deval Shah : 2024CS10132

Instructor

Prof. Piyush Kumar Sharma

Contents

1	Overview	3
2	Implementation	3
2.1	System Architecture	3
2.2	Concurrency and I/O	3
2.3	TCP Message Framing	4
2.4	Output Buffering	4
2.5	Client I/O	4
2.6	Order Matching	4
3	Experiment 1: Listening and Connected Sockets	4
3.1	Objective	4
3.2	Procedure	5
3.3	Observation	5
3.4	Answer	5
4	Experiment 2: Observing TCP Connection States	5
4.1	Objective	5
4.2	Procedure	6
4.3	Observation	6
4.4	Answer	7
5	Experiment 3: TCP as a Byte Stream	7
5.1	Objective	7
5.2	Procedure	7
5.3	Answer	8
6	Experiment 4: One Client Should Not Stall the Others	8
6.1	Objective	8
6.2	Procedure	8
6.3	Answer	9
7	Experiment 5: Multiple Simultaneous Connections	9
7.1	Objective	9
7.2	Procedure	9
7.3	Observation	9
7.4	Answer	10
8	Experiment 6: Abrupt Client Termination	10
8.1	Objective	10

8.2	Procedure	10
8.3	Observation	11
8.4	Answer	12
9	Experiment 7: Slow Receiver and Backpressure	12
9.1	Objective	12
9.2	Procedure	12
9.3	Observation	12
9.4	Answer	13
10	Experiment 8: Unexpected Client Disconnection	13
10.1	Objective	13
10.2	Procedure	14
10.3	Observation	14
10.4	Answer	15
11	Implementation Testing	15
12	Implementation Decisions	16
12.1	Concurrency and I/O Approach	16
12.2	Other Significant Network Design Decisions	16
13	Conclusion	16
14	Bonus: Connection Scalability and I/O Design	17
14.1	Implementation Detail and Deliverables	17
14.2	OS Bottlenecks Analysis	17
14.3	Experimental Proof	17
15	Pseudocode	18
15.1	common.py	18
15.2	server.py	19
15.3	trader.py	20
15.4	market_data.py	20

1 Overview

This assignment implements and investigates a simplified trading exchange using TCP sockets.

The system consists of:

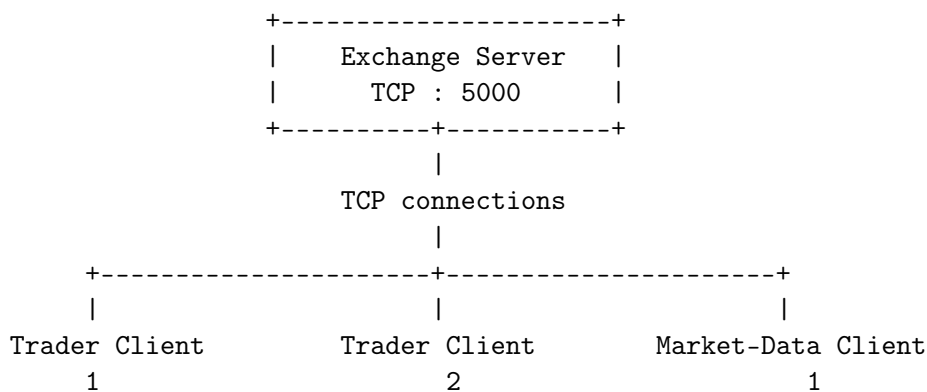
- an Exchange Server,
- Trader Clients, and
- Market-Data Clients.

All components run as separate processes inside a FreeBSD virtual machine. Communication between the clients and the server takes place using TCP sockets.

2 Implementation

2.1 System Architecture

The complete system follows the structure:

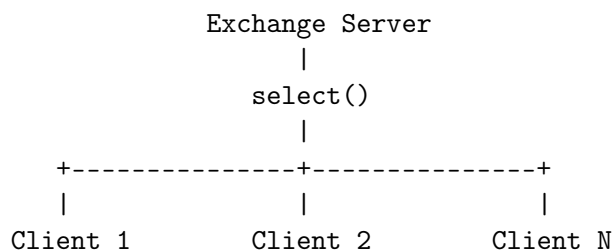


The server maintains one TCP connection for each client.

2.2 Concurrency and I/O

The Exchange Server uses a single process with `select()` to monitor the listening socket and all connected client sockets.

Instead of creating a separate thread or process for every client, the server waits for sockets that are ready for reading or writing.



This allows an idle client to remain connected without preventing the server from servicing other clients.

2.3 TCP Message Framing

TCP provides a continuous byte stream rather than application-level message boundaries.

Therefore, one `send()` does not necessarily correspond to one `recv()`.

The protocol uses the newline character:

`\n`

as the application-level message delimiter.

A `LineBuffer` stores incomplete data until a complete line is received.

2.4 Output Buffering

Server sockets are non-blocking. When a response cannot be sent immediately, the response is stored in a per-client output buffer.

The socket is added to the writable set of `select()` and the pending data is sent when the socket becomes writable.

This prevents one slow receiver from blocking the entire server.

2.5 Client I/O

Trader and Market-Data clients use `select()` to monitor:

- the TCP socket, and
- standard input.

Thus, clients can receive asynchronous notifications from the server even while waiting for user input.

2.6 Order Matching

The server maintains outstanding orders for the instruments JNST and IMCT.

A BUY order and SELL order can match when:

- they refer to the same instrument, and
- they have exactly the same price.

The traded quantity is:

$$q_{\text{trade}} = \min(q_{\text{buy}}, q_{\text{sell}})$$

If an order is only partially filled, its remaining quantity stays in the order book.

3 Experiment 1: Listening and Connected Sockets

3.1 Objective

Identify the listening socket and the connected sockets after establishing a TCP connection.

3.2 Procedure

The experiment was executed using:

```
python3 experiment.py 1
```

The TCP sockets were inspected using:

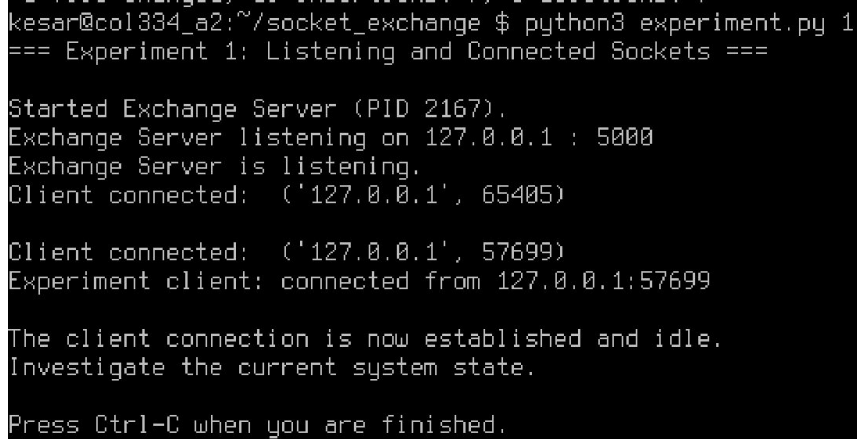
```
sockstat -4 -P tcp | grep 5000
```

3.3 Observation

The server has a listening socket bound to:

127.0.0.1:5000

After a client connects, `accept()` creates a separate connected socket for that client.



```
kesar@col334_a2:~/socket_exchange $ python3 experiment.py 1
=== Experiment 1: Listening and Connected Sockets ===

Started Exchange Server (PID 2167).
Exchange Server listening on 127.0.0.1 : 5000
Exchange Server is listening.
Client connected: ('127.0.0.1', 65405)

Client connected: ('127.0.0.1', 57699)
Experiment client: connected from 127.0.0.1:57699

The client connection is now established and idle.
Investigate the current system state.

Press Ctrl-C when you are finished.
```

Figure 1: Experiment 1 running in the FreeBSD VM.



```
[kesar@col334_a2:~/socket_exchange $ sockstat -4 -P tcp | grep 5000
kesar  python3.12  2167  3  tcp4      127.0.0.1:5000      *:*
kesar  python3.12  2167  4  tcp4      127.0.0.1:5000      127.0.0.1:57699
kesar  python3.12  2166  3  tcp4      127.0.0.1:57699     127.0.0.1:5000
```

Figure 2: TCP sockets observed using sockstat.

3.4 Answer

The listening socket waits for new connections, while each accepted socket represents an actual TCP connection with one client. Therefore, the server has one listening socket but can have many connected sockets.

4 Experiment 2: Observing TCP Connection States

4.1 Objective

Observe TCP states during connection establishment, active communication, and termination.

4.2 Procedure

The experiment was executed using:

```
python3 experiment.py 2
```

The connection was inspected using:

```
netstat -an -p tcp | grep 5000
```

4.3 Observation

While the client was connected, the connection appeared as:

```
ESTABLISHED
```

The server's listening socket remained in:

```
LISTEN
```

After the client connection was closed, the established connection disappeared while the listening socket remained.

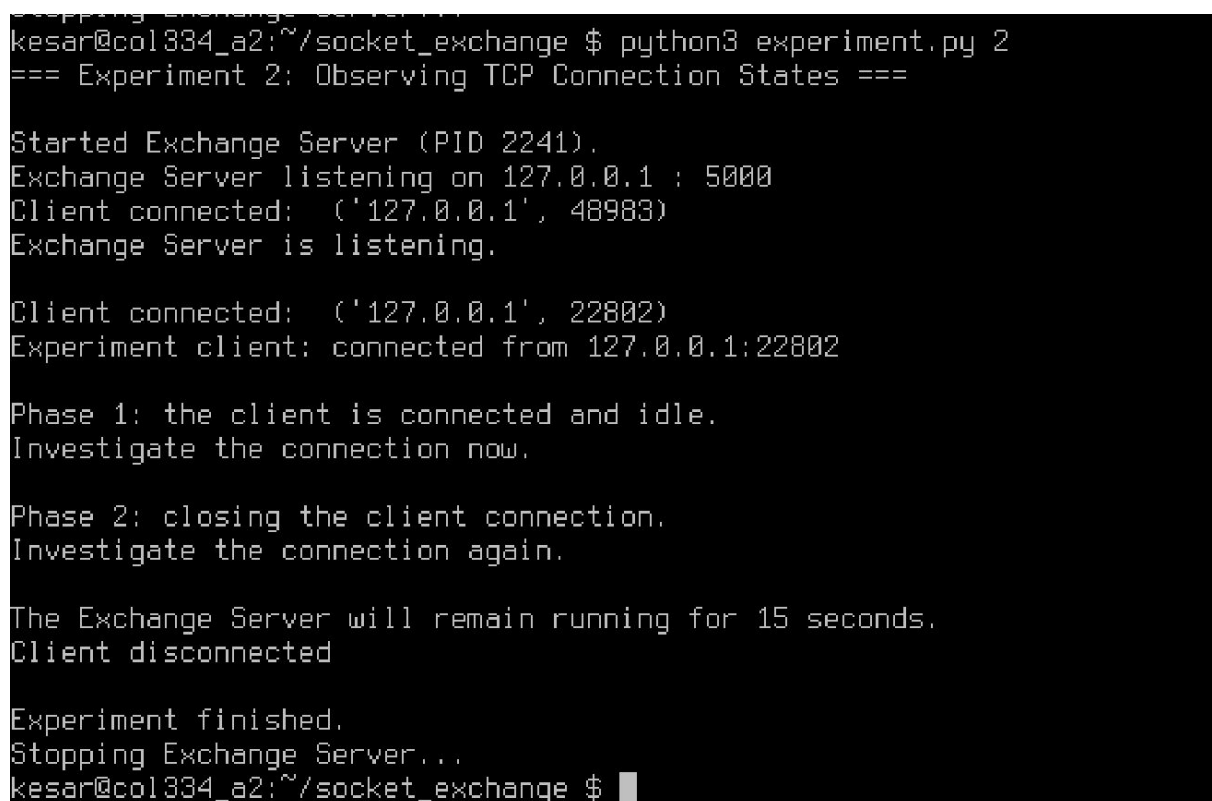
A terminal window with a black background and white text. The text shows the execution of a Python script named 'experiment.py' with argument '2'. The script starts by printing 'Stopping Exchange Server...' and then 'kesar@col334_a2:~/socket_exchange \$ python3 experiment.py 2'. It then prints '=== Experiment 2: Observing TCP Connection States ==='. The script starts an 'Exchange Server (PID 2241)' listening on '127.0.0.1 : 5000'. It then shows two client connections: one from '127.0.0.1' with port '48983' and another from '127.0.0.1' with port '22802'. The script then enters 'Phase 1: the client is connected and idle. Investigate the connection now.' followed by 'Phase 2: closing the client connection. Investigate the connection again.' It then states 'The Exchange Server will remain running for 15 seconds.' and 'Client disconnected'. Finally, it prints 'Experiment finished.' and 'Stopping Exchange Server...' before returning to the prompt 'kesar@col334_a2:~/socket_exchange \$'.

Figure 3: Experiment 2 showing the connection phases.

```

kesar@col334_a2:~/socket_exchange $ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      127.0.0.1.22802    ESTABLISHED
tcp4      0      0 127.0.0.1.22802    127.0.0.1.5000    ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                LISTEN
kesar@col334_a2:~/socket_exchange $ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      *.*                LISTEN
kesar@col334_a2:~/socket_exchange $ netstat -an -p tcp | grep 5000
kesar@col334_a2:~/socket_exchange $ █

```

Figure 4: TCP states observed using netstat.

4.4 Answer

The client-server connection remains in the ESTABLISHED state while communication is active. The listening socket remains in LISTEN throughout, allowing the server to accept new connections.

5 Experiment 3: TCP as a Byte Stream

5.1 Objective

Investigate whether separate TCP writes preserve application-level message boundaries.

5.2 Procedure

The experiment was executed using:

```
python3 experiment.py 3
```

One application-level message was divided into four TCP writes:

```

6 bytes
10 bytes
7 bytes
1 byte

```

The server ultimately reconstructed:

```
LOGIN experiment_trader
```

```

socket_exchange_server.py
kesar@col334_a2:~/socket_exchange $ python3 experiment.py 3
=== Experiment 3: TCP as a Byte Stream ===

Started Exchange Server (PID 2253).
Exchange Server listening on 127.0.0.1 : 5000
Client connected: ('127.0.0.1', 47563)
Exchange Server is listening.

Experiment client: connected from 127.0.0.1:38293

Sending one application-level message using several separate TCP writes...

Client connected: ('127.0.0.1', 38293)
Sent 6 bytes.
Sent 10 bytes.
Sent 7 bytes.
Sent 1 byte.
Received: LOGIN experiment_trader

Investigate what the Exchange Server received.
Press Ctrl-C when you are finished.
█

```

Figure 5: One application message sent using several TCP writes.

5.3 Answer

TCP provides a continuous byte stream and does not preserve application-level message boundaries. A message can therefore be split across multiple `recv()` calls. The server solves this using newline-based message framing.

6 Experiment 4: One Client Should Not Stall the Others

6.1 Objective

Determine whether an idle client can prevent the server from servicing another active client.

6.2 Procedure

The experiment was executed using:

```
python3 experiment.py 4
```

Client 1 sent an incomplete application message and then remained silent.

Client 2 connected and sent:

```
LOGIN active_client
```

The server immediately returned:

```
OK
```

with an observed elapsed time of approximately 0.000 seconds.

```
kesar@col334_a2:~/socket_exchange $ python3 experiment.py 4
=== Experiment 4: One Client Should Not Stall the Others ===

Started Exchange Server (PID 2263).
Exchange Server listening on 127.0.0.1 : 5000
Exchange Server is listening.

Client connected: ('127.0.0.1', 11387)
Client 1: connected from 127.0.0.1:48984

Client 1 will send an incomplete application message.
Client connected: ('127.0.0.1', 48984)
Client 1 will now remain silent.

Client 2: connected from 127.0.0.1:23053
Client 2 will send a complete application message.
Client connected: ('127.0.0.1', 23053)
Received: LOGIN active_client

Client 2 response: 'OK'
Elapsed time: 0.000 seconds

Investigate the behavior of both TCP connections.
Press Ctrl-C when you are finished.
```

Figure 6: Experiment 4 with an idle and an active client.

```
kesar@col334_a2:~/socket_exchange $ netstat -an -p tcp | grep 5000
tcp4      0      0 127.0.0.1.5000      127.0.0.1.23053      ESTABLISHED
tcp4      0      0 127.0.0.1.23053     127.0.0.1.5000       ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.48984      ESTABLISHED
tcp4      0      0 127.0.0.1.48984     127.0.0.1.5000       ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
kesar@col334_a2:~/socket_exchange $
```

Figure 7: Both TCP connections observed using netstat.

6.3 Answer

The idle Client 1 does not block Client 2. The server uses `select()` instead of blocking indefinitely on one client's `recv()`. Hence, the server can service another socket whenever it becomes ready.

7 Experiment 5: Multiple Simultaneous Connections

7.1 Objective

Investigate how the server handles multiple simultaneous TCP connections and identify sockets that are ready for I/O.

7.2 Procedure

The experiment was executed using:

```
python3 experiment.py 5
```

7.3 Observation

Five simultaneous client connections were established with the Exchange Server. Clients 1, 3, and 5 actively communicated, while Clients 2 and 4 remained idle. Upon inspecting the sockets with `netstat`, we explicitly observed three connections with a receive queue (Recv-Q) of exactly 3 bytes, representing unread responses. The other two idle connections maintained a Recv-Q of 0.

```
To change this login announcement, see motd(5).
You can press Ctrl-L while in the shell to clear the screen.
preksha@preksha:~ $ netstat -an | grep 5000
tcp4      0      0 127.0.0.1.5000      127.0.0.1.36846      ESTABLISHED
tcp4      3      0 127.0.0.1.36846     127.0.0.1.5000       ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.13293      ESTABLISHED
tcp4      0      0 127.0.0.1.13293     127.0.0.1.5000       ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.50080      ESTABLISHED
tcp4      3      0 127.0.0.1.50080     127.0.0.1.5000       ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.39109      ESTABLISHED
tcp4      0      0 127.0.0.1.39109     127.0.0.1.5000       ESTABLISHED
tcp4      0      0 127.0.0.1.5000      127.0.0.1.14588      ESTABLISHED
tcp4      3      0 127.0.0.1.14588     127.0.0.1.5000       ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
```

Figure 8: Experiment 5 socket investigation.

```

root@prekshn:/home/prekshn/socket_exchange # python3 experiment.py 5
=== Experiment 5: Multiple Clients and I/O Multiplexing (Optional) ===

Started Exchange Server (PID 4364).
Exchange Server listening on 127.0.0.1 : 5000
Exchange Server is listening.

Creating five simultaneous TCP connections...

Client connected: ('127.0.0.1', 29048)
Client 1: connected from 127.0.0.1:14588
Client 2: connected from 127.0.0.1:39109
Client connected: ('127.0.0.1', 14588)
Client connected: ('127.0.0.1', 39109)
Client connected: ('127.0.0.1', 50080)
Client 3: connected from 127.0.0.1:50080
Client connected: ('127.0.0.1', 13293)
Client 4: connected from 127.0.0.1:13293
Client connected: ('127.0.0.1', 36846)
Client 5: connected from 127.0.0.1:36846

Clients 1, 3, and 5 will send application messages.
Clients 2 and 4 will remain idle.

Client 1: sent application data.
Received: LOGIN client_1
Client 3: sent application data.
Received: LOGIN client_3
Client 5: sent application data.
Received: LOGIN client_5

The experiment is now in the observation phase.
Investigate which connections were active.
Press Ctrl-C when you are finished.
Stopping Exchange Server...

```

Figure 9: Experiment 5 running in the FreeBSD VM.

7.4 Answer

The connections for Clients 1, 3, and 5 are the ones communicating. The evidence from the running system is the non-zero Recv-Q (receive queue) values for these specific active connections observed via `netstat`. A multiplexing system call like `select()` will identify sockets as readable when there is data pending in these queues.

8 Experiment 6: Abrupt Client Termination

8.1 Objective

Investigate how the server detects an abruptly terminated client connection.

8.2 Procedure

The experiment was executed using:

```
python3 experiment.py 6
```

8.3 Observation

During the orderly shutdown (Part A), the client sent a FIN packet, closing the connection gracefully (which we observed as Flags [F.] in tcpdump). During the abrupt termination (Part B), the client process was killed unexpectedly, resulting in an RST (Reset) packet being sent over the network, which was explicitly captured as Flags [R.] using tcpdump.

```
Started Exchange Server (PID 4378).
Exchange Server listening on 127.0.0.1 : 5000
Exchange Server is listening.

Part A: orderly connection termination (FIN).
Client connected: ('127.0.0.1', 63001)
Orderly client: connected from 127.0.0.1:51360
The client will shut down its sending direction.
Client connected: ('127.0.0.1', 51360)
Investigate the TCP traffic and connection state.
Client disconnected

Part B: abortive connection termination (RST).
Abrupt client: connected from 127.0.0.1:47830
The client will now close abortively.
Investigate the TCP traffic and connection state.

The Exchange Server will remain running for 15 seconds.
Client connected: ('127.0.0.1', 47830)

Experiment finished.
Stopping Exchange Server...
root@preksha:/home/preksha/socket_exchange # exit
preksha@preksha:~/socket_exchange $ python3 experiment.py 6
=== Experiment 6: FIN vs. RST: Orderly and Abrupt Connection Termination ===

Started Exchange Server (PID 4407).
Exchange Server listening on 127.0.0.1 : 5000
Exchange Server is listening.

Part A: orderly connection termination (FIN).
Client connected: ('127.0.0.1', 27308)
Orderly client: connected from 127.0.0.1:64992
The client will shut down its sending direction.
Investigate the TCP traffic and connection state.
Client connected: ('127.0.0.1', 64992)
Client disconnected

Part B: abortive connection termination (RST).
Abrupt client: connected from 127.0.0.1:41434
The client will now close abortively.
Investigate the TCP traffic and connection state.

The Exchange Server will remain running for 15 seconds.
Client connected: ('127.0.0.1', 41434)

Experiment finished.
Stopping Exchange Server...
```

Figure 10: Experiment 6 running in the FreeBSD VM.

```
root@preksha:/home/preksha # tcpdump -i lo0 port 5000
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on lo0, link-type NULL (BSD loopback), snapshot length 262144 bytes
15:47:35.438784 IP localhost.10354 > localhost.5000: Flags [S], seq 162397782, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 3932388588 ecr 0], length 0
15:47:35.438832 IP localhost.5000 > localhost.10354: Flags [R.], seq 0, ack 162397783, win 0, length 0
15:47:35.543430 IP localhost.27308 > localhost.5000: Flags [S], seq 253884777, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2907833190 ecr 0], length 0
15:47:35.543515 IP localhost.5000 > localhost.27308: Flags [S.], seq 516006113, ack 253884778, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 83664133 ecr 2907833190], length 0
15:47:35.543529 IP localhost.27308 > localhost.5000: Flags [.], ack 1, win 320, options [nop,nop,TS val 2907833190 ecr 83664133], length 0
15:47:35.543619 IP localhost.27308 > localhost.5000: Flags [R.], seq 1, ack 1, win 0, options [nop,nop,TS val 2907833190 ecr 83664133], length 0
15:47:35.543784 IP localhost.64992 > localhost.5000: Flags [S], seq 2807396762, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 831277691 ecr 0], length 0
15:47:35.543714 IP localhost.5000 > localhost.64992: Flags [S.], seq 1238291382, ack 2807396763, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 521205731 ecr 831277691], length 0
15:47:35.543719 IP localhost.64992 > localhost.5000: Flags [.], ack 1, win 320, options [nop,nop,TS val 831277691 ecr 521205731], length 0
15:47:35.543763 IP localhost.64992 > localhost.5000: Flags [F.], seq 1, ack 1, win 320, options [nop,nop,TS val 831277691 ecr 521205731], length 0
15:47:35.543765 IP localhost.5000 > localhost.64992: Flags [.], ack 2, win 320, options [nop,nop,TS val 521205731 ecr 831277691], length 0
15:47:35.543793 IP localhost.5000 > localhost.64992: Flags [F.], seq 1, ack 2, win 320, options [nop,nop,TS val 521205731 ecr 831277691], length 0
15:47:35.543823 IP localhost.64992 > localhost.5000: Flags [.], ack 2, win 320, options [nop,nop,TS val 831277691 ecr 521205731], length 0
15:47:45.546248 IP localhost.41434 > localhost.5000: Flags [S], seq 2205922726, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2379050198 ecr 0], length 0
15:47:45.546399 IP localhost.5000 > localhost.41434: Flags [S.], seq 2394622576, ack 2205922727, win 65535, options [mss 16344,nop,wscale 8,sackOK,TS val 2379050198 ecr 994097355], length 0
15:47:45.546436 IP localhost.41434 > localhost.5000: Flags [.], ack 1, win 320, options [nop,nop,TS val 2379050198 ecr 994097355], length 0
15:47:45.546695 IP localhost.41434 > localhost.5000: Flags [R.], seq 1, ack 1, win 0, options [nop,nop,TS val 2379050198 ecr 994097355], length 0
^C
17 packets captured
17 packets received by filter
0 packets dropped by kernel
root@preksha:/home/preksha #
```

Figure 11: Experiment 6 network/socket investigation.


```

0 packets dropped by kernel
[root@preksha:/home/preksha # netstat -an | grep 5000
tcp4      0      35 127.0.0.1.5000      127.0.0.1.36213      ESTABLISHED
tcp4      0      0 127.0.0.1.36213      127.0.0.1.5000      ESTABLISHED
tcp4      0      37 127.0.0.1.5000      127.0.0.1.41362      ESTABLISHED
tcp4      0      0 127.0.0.1.41362      127.0.0.1.5000      ESTABLISHED
tcp4      0      34 127.0.0.1.5000      127.0.0.1.28551      ESTABLISHED
tcp4    3213      0 127.0.0.1.28551      127.0.0.1.5000      ESTABLISHED
tcp4      0      34 127.0.0.1.5000      127.0.0.1.55621      ESTABLISHED
tcp4      0      0 127.0.0.1.55621      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[root@preksha:/home/preksha # netstat -an | grep 5000
tcp4      0      35 127.0.0.1.5000      127.0.0.1.36213      ESTABLISHED
tcp4      0      0 127.0.0.1.36213      127.0.0.1.5000      ESTABLISHED
tcp4      0      37 127.0.0.1.5000      127.0.0.1.41362      ESTABLISHED
tcp4      0      0 127.0.0.1.41362      127.0.0.1.5000      ESTABLISHED
tcp4      0      17 127.0.0.1.5000      127.0.0.1.28551      ESTABLISHED
tcp4    7854      0 127.0.0.1.28551      127.0.0.1.5000      ESTABLISHED
tcp4      0      17 127.0.0.1.5000      127.0.0.1.55621      ESTABLISHED
tcp4      0      0 127.0.0.1.55621      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[root@preksha:/home/preksha # netstat -an | grep 5000
tcp4      0      36 127.0.0.1.5000      127.0.0.1.36213      ESTABLISHED
tcp4      0      0 127.0.0.1.36213      127.0.0.1.5000      ESTABLISHED
tcp4      0      38 127.0.0.1.5000      127.0.0.1.41362      ESTABLISHED
tcp4      0      0 127.0.0.1.41362      127.0.0.1.5000      ESTABLISHED
tcp4      0      34 127.0.0.1.5000      127.0.0.1.28551      ESTABLISHED
tcp4    9945      0 127.0.0.1.28551      127.0.0.1.5000      ESTABLISHED
tcp4      0      34 127.0.0.1.5000      127.0.0.1.55621      ESTABLISHED
tcp4      0      0 127.0.0.1.55621      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[root@preksha:/home/preksha # netstat -an | grep 5000
tcp4      0      36 127.0.0.1.5000      127.0.0.1.36213      ESTABLISHED
tcp4      0      0 127.0.0.1.36213      127.0.0.1.5000      ESTABLISHED
tcp4      0      38 127.0.0.1.5000      127.0.0.1.41362      ESTABLISHED
tcp4      0      0 127.0.0.1.41362      127.0.0.1.5000      ESTABLISHED
tcp4      0      34 127.0.0.1.5000      127.0.0.1.28551      ESTABLISHED
tcp4   13498      0 127.0.0.1.28551      127.0.0.1.5000      ESTABLISHED
tcp4      0      34 127.0.0.1.5000      127.0.0.1.55621      ESTABLISHED
tcp4      0      0 127.0.0.1.55621      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN
[root@preksha:/home/preksha # netstat -an | grep 5000
tcp4      0      36 127.0.0.1.5000      127.0.0.1.36213      ESTABLISHED
tcp4      0      0 127.0.0.1.36213      127.0.0.1.5000      ESTABLISHED
tcp4      0      38 127.0.0.1.5000      127.0.0.1.41362      ESTABLISHED
tcp4      0      0 127.0.0.1.41362      127.0.0.1.5000      ESTABLISHED
tcp4      0      34 127.0.0.1.5000      127.0.0.1.28551      ESTABLISHED
tcp4   15623      0 127.0.0.1.28551      127.0.0.1.5000      ESTABLISHED
tcp4      0      34 127.0.0.1.5000      127.0.0.1.55621      ESTABLISHED
tcp4      0      0 127.0.0.1.55621      127.0.0.1.5000      ESTABLISHED
tcp4      0      0 127.0.0.1.5000      *.*                  LISTEN

```

Figure 13: Experiment 7 socket/network investigation.

9.4 Answer

As the slow client stops reading, its TCP receive window shrinks to zero, which blocks the server from transmitting more data. This causes the server's send buffer for that client to fill up. Since the server uses non-blocking I/O with `select()`, a `send()` to the slow client returns `EWOULDBLOCK` or `EAGAIN`, allowing the server to gracefully skip it and continue servicing other clients without stalling.

10 Experiment 8: Unexpected Client Disconnection

10.1 Objective

Investigate how the server detects an unexpected Market-Data Client disconnection while communicating with other clients.

10.2 Procedure

The experiment was executed using:

```
python3 experiment.py 8
```

10.3 Observation

When the Market-Data Client disappeared unexpectedly, the server’s TCP connection to the client remained stuck in the ESTABLISHED state. However, because the client was unresponsive, unacknowledged data began accumulating in the server’s Send-Q (we explicitly observed queues holding 17, 35, and 37 bytes). Because the server did not receive an orderly FIN or RST, it continued attempting to transmit.

```
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Received: BUY JNST 1 238  
Received: SELL JNST 1 238  
Experiment finished.  
Stopping Exchange Server..  
preksha@preksha:~/socket_e
```

Figure 14: Experiment 8 running in the FreeBSD VM.

```
=====
```

tcp4	0	35	127.0.0.1.5000	127.0.0.1.48713	ESTABLISHED
tcp4	35	0	127.0.0.1.48713	127.0.0.1.5000	ESTABLISHED
tcp4	0	37	127.0.0.1.5000	127.0.0.1.12326	ESTABLISHED
tcp4	0	0	127.0.0.1.12326	127.0.0.1.5000	ESTABLISHED
tcp4	0	17	127.0.0.1.5000	127.0.0.1.12182	ESTABLISHED
tcp4	17	0	127.0.0.1.12182	127.0.0.1.5000	ESTABLISHED
tcp4	0	0	127.0.0.1.5000	.*.*	LISTEN

```
=====
```

Figure 15: Experiment 8 socket/network investigation.

10.4 Answer

When the client disappears unexpectedly, the TCP connection remains active on the server, but unacknowledged data builds up in its send buffer. The network-level evidence is the presence of data trapped in the server's Send-Q alongside repeated TCP packet retransmissions without receiving any ACKs. The server eventually detects the failure when the TCP retransmission timer expires, causing the socket to return an ETIMEDOUT error.

11 Implementation Testing

A separate sanity-check script was used to test important protocol and implementation properties.

The tests covered:

1. required launchers,
2. basic TCP connection and LOGIN,
3. fragmented TCP message reconstruction,
4. rejection of zero quantity/price,
5. role restrictions,
6. order matching,
7. partial fills and market-data notifications,
8. CANCEL,
9. orders surviving trader disconnection, and
10. ten simultaneous idle connections.

The sanity-check run concluded with:

ALL SANITY TESTS PASSED

```
preksha@preksha:~/socket_exchange $ python3 sanity_check.py
PASS 1: required launchers exist and are executable
Server started
PASS 2: basic TCP connection + LOGIN
PASS 3: fragmented TCP message reconstructed
PASS 4: zero quantity/price rejected
PASS 5: role restrictions
PASS 6: matching + BOUGHT/SOLD
PASS 7: partial fills + market-data TRADE
PASS 8: CANCEL
PASS 9: orders survive trader disconnect
PASS 10: 10 simultaneous idle connections

ALL SANITY TESTS PASSED
```

Figure 16: Implementation sanity checks.

12 Implementation Decisions

The development of the Exchange Server required several key design choices to ensure robust and concurrent network communication.

12.1 Concurrency and I/O Approach

The Exchange Server handles multiple clients using a single-threaded I/O multiplexing approach via the `select()` system call.

Why this approach was chosen: We chose `select()` because it provides a highly portable and deterministic way to handle multiple simultaneous connections. It avoids the heavy context-switching overhead and complex synchronization issues (such as race conditions and mutex locks) associated with multi-threading. By using `select()`, the server can efficiently monitor the listening socket alongside all connected client sockets, waking up to service them strictly when they are ready for reading or writing.

12.2 Other Significant Network Design Decisions

- **Non-blocking Sockets and Output Buffering:** To guarantee that one problematic client cannot stall the entire exchange, all client sockets are set to non-blocking mode. If a client becomes a slow receiver and its TCP window fills up, a `send()` operation would normally block. Instead, our server queues the unacknowledged data in a per-client output buffer, adds the socket to the writable set in `select()`, and flushes the remaining data only when the socket becomes writable again.
- **TCP Message Framing:** Because TCP is a continuous byte stream, data boundaries are not preserved; a single `recv()` might return half a message or multiple concatenated messages. To solve this, we implemented a strict newline-character (`\n`) delimiter protocol. The server maintains a custom `LineBuffer` for every client that accumulates incoming bytes and safely parses them into discrete, complete application-level commands before execution.

13 Conclusion

The implementation demonstrates the use of TCP sockets for a concurrent client-server application.

The experiments show that:

- a listening socket is different from an accepted connected socket;
- established TCP connections are independent of the listening socket;
- TCP does not preserve application-level message boundaries;
- newline-based framing is required to reconstruct messages correctly;
- an idle client should not block other clients;
- `select()` provides a simple mechanism for I/O multiplexing;
- non-blocking output handling prevents a slow client from blocking the complete server.

14 Bonus: Connection Scalability and I/O Design

14.1 Implementation Detail and Deliverables

```
Initialize:
    create IPv4 socket (127.0.0.1:5000)
    create IPv6 socket (::1:5000)
    kq = create kqueue object
    register sockets to kq for READ

Event Loop:
    events = kq.control(wait for events)
    for each event:
        if server socket is readable:
            accept new client
            register client socket to kq
        else if client socket is readable:
            receive data
            if data exists:
                process orders
```

14.2 OS Bottlenecks Analysis

- **File Descriptor Limit (1024):** select crashed. → **Fix:** Migrated to kqueue.
- **Ephemeral Port Exhaustion (55,000):** Hit Errno 49 on single IP. → **Fix:** Bound server to both IPv4 and IPv6 to double the port pool to 110,000+.
- **Kernel File Descriptor Limit:** Hit Errno 23. → **Fix:** Increased kern.maxfiles and kern.maxfilesperproc via sysctl as root.
- **Kernel Socket Limit:** Hit Errno 55 (No buffer space). → **Fix:** Increased kern.ipc.maxsockets to 300,000.

14.3 Experimental Proof

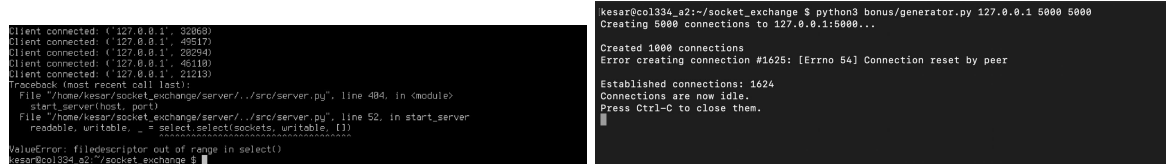


Figure 17: File Descriptor limits hit during the 60,000 connections phase.

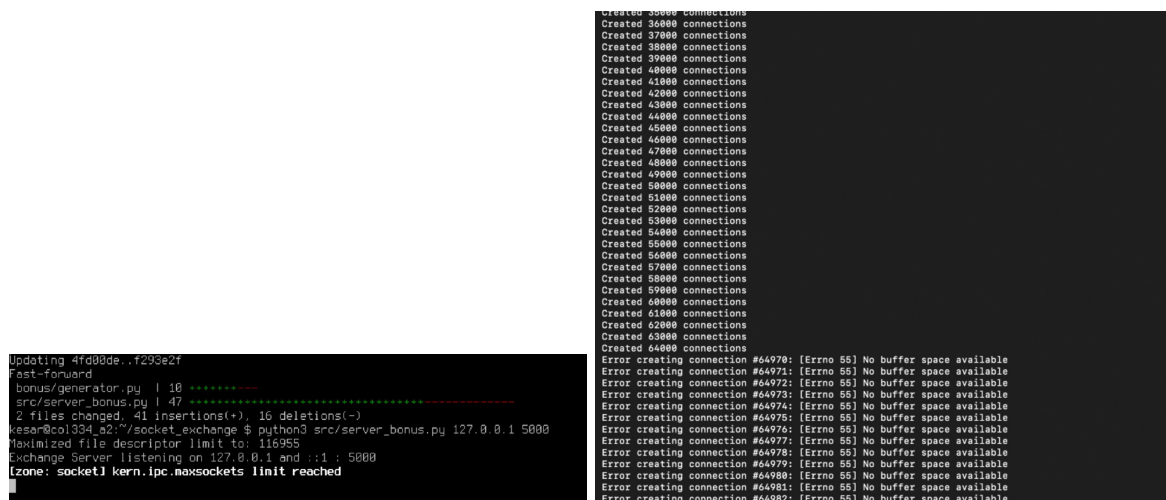


Figure 18: Global socket memory exhaustion before reaching 70,000.

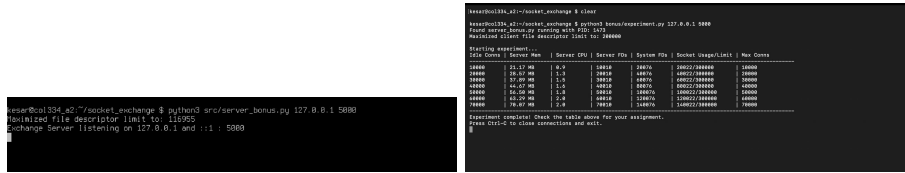


Figure 19: Successful generation of 70,000 concurrent idle connections.

Idle Conns	Server Mem	CPU %	Server FDs	System FDs	Socket Usage / Limit
10,000	21.17 MB	0.9%	10,010	20,076	20,022 / 300,000
20,000	28.57 MB	1.3%	20,010	40,076	40,022 / 300,000
30,000	37.89 MB	1.5%	30,010	60,076	60,022 / 300,000
40,000	44.67 MB	1.6%	40,010	80,076	80,022 / 300,000
50,000	56.50 MB	1.8%	50,010	100,076	100,022 / 300,000
60,000	63.29 MB	2.0%	60,010	120,076	120,022 / 300,000
70,000	70.07 MB	2.0%	70,010	140,076	140,022 / 300,000

Table 1: System metrics while maintaining 70,000 established TCP connections on FreeBSD.

15 Pseudocode

The following pseudocode summarizes the three source files. It is intentionally compact and uses a console-style font.

15.1 common.py

```
INSTRUMENTS = {JNST, IMCT}
INT_MAX = 2147483647

LineBuffer:
    buffer = empty bytes

    add(data):
        buffer += data
        while '\n' in buffer:
            line, buffer = split at first '\n'
            remove trailing '\r' if present
            decode line
            append line to messages
        return messages

encode_line(message):
    return (message + '\n').encode()

send_line(sock, message):
    sock.sendall(encode_line(message))

valid_instrument(x):
    return x in INSTRUMENTS

valid_int(x):
    try:
        n = int(x)
        return 0 <= n <= INT_MAX
    except:
        return False

valid_order_id(x):
    try:
        n = int(x)
        return 0 <= n <= INT_MAX
    except:
        return False
```

15.2 server.py

```
create TCP listening socket
set SO_REUSEADDR
bind to host:port
listen
set listening socket non-blocking

while server is running:

    readable = select(read sockets)
    writable = select(sockets with pending output)

    if listening socket is readable:
        accept new client
        set client socket non-blocking
        create ClientState
        add client to socket list

    for each readable client:
        recv data

        if connection closed:
            remove client
            close socket
            continue

        add data to client's LineBuffer

        for each complete message:
            process_message(client, message)

            LOGIN:
                validate username
                register trader

            BUY / SELL:
                validate role, instrument, quantity, price
                create order and assign ID
                add order to book
                send ORDER_ACCEPTED
                match against opposite book

            CANCEL:
                validate order ID and ownership
                remove outstanding order
                send ORDER_CANCELLED

            SUBSCRIBE / UNSUBSCRIBE:
                validate market-data role/instrument
                update subscriptions
                send OK

            QUIT:
                close client connection

    for each writable client:
        send pending output
        remove bytes that were successfully sent

match_orders(order):
    find opposite order with same instrument and price

    while matching order exists and quantity remains:
        traded_qty = min(buy_qty, sell_qty)

        send BOUGHT to buyer
        send SOLD to seller
        broadcast TRADE to subscribers

        reduce both quantities
        remove fully-filled orders
```

15.3 trader.py

```
create TCP socket
connect(host, port)
send LOGIN username

while running:

    readable = select(socket, stdin)

    if socket readable:
        recv data
        if disconnected:
            exit
        add data to LineBuffer
        print every complete server message

    if stdin readable:
        read command
        ignore empty command
        send command + '\n'

        if command == QUIT:
            exit

on Ctrl-C:
    try to send QUIT

close socket
```

15.4 market_data.py

```
validate instrument

create TCP socket
connect(host, port)
send SUBSCRIBE instrument

while running:

    readable = select(socket, stdin)

    if socket readable:
        recv data
        if disconnected:
            exit
        add data to LineBuffer
        print every complete market-data message

    if stdin readable:
        read command
        ignore empty command
        send command + '\n'

        if command == QUIT:
            exit

on Ctrl-C:
    try to send QUIT

close socket
```